```
    return $ret;
}

Output:
Array
(
    [Alice] => 3
    [Bob] => 5
    [Russell] => 1
)
```

A recursive function is simply a function that calls itself. When traversing `$struct` with our user-defined function `recursive_array_count()`, we test if the current element in the `foreach` loop is an array. If it is, we call `recursive_array_count()` again and pass the current element (and a reference to `$ret` so we can keep a tally of the counts in the second call) so the loop can iterate for the sub-array before returning control to the first call of `recursive_array_count()`. This will happen every time our function encounters an array, so be aware of potential pitfalls such as circular references, very large arrays, and so on.

For example, calling a recursive function on the following has some nasty side-effects:

```
$struct =[
    ['Alice', 'Bob', 'Bob'],
    ['Alice', 'Bob', 'Bob'],
    ['Alice', 'Russell', 'Bob'],
    &$struct,
];
```

Here `$struct` contains a reference to itself. If `recursive_array_count()` was called on this structure, it would continue to call itself every time it hits the reference, which contains yet another reference, and so on. Execution can never return to the calling program and eventually Something Bad will happen. This could be stack overflow, exceeding `memory_limit`, or hitting a recursion limit if using a tool like `xdebug`. The ultimate outcome depends on your system configuration, but none are desirable.